Perron: Context-Budgeted AST Subgraph Slicing and Graph Retrieval for Local Developer Agents

Sultan Haikal
yvliet@users.noreply.github.com

October 2026

Abstract

Local developer agents must find the code an issue refers to within a context budget of a few thousand tokens ($K \leq 4{,}096$). Graph retrieval is the usual way to reach code that the issue never names, but repository call graphs are scale-free, and Personalized PageRank (PPR) moves relevance into a small set of utility hubs. I measure this on all 300 SWE-bench Lite instances under a pre-registered 50/100/150 split. At damping $\beta = 0.85$, six diffusion retrievers (standard PPR, Aider repo maps, HippoRAG-style prior scaling, query-reweighted PPR, a hub blocklist and a degree-discounted matrix) retrieve none of the 118 held-out callable defect targets; the median in-degree of standard PPR's top-10 symbols is 135.0, against 3.0 for the gold targets. **Perron** divides the converged query distribution by a repository-wide stationary baseline, $\pi_q/(\pi_g + \epsilon)^\gamma$, which recovers 17/118 (14.4%) Function Acc@10 while the CSR transition matrix stays static and memory-mapped; a Neumann-series argument shows why pre-walk prior scaling cannot do the same. BM25 remains stronger (27/118, McNemar $p = 0.052$), but the two retrievers hit different targets: 5 of Perron's 6 exclusive hits lie outside BM25's top 20. No fusion I tested converts this into a top-10 gain (RRF 25/118 vs. 27/118), and the union of both top-10 lists (33/118) is matched by BM25 given the same 20 slots (31/118, $p = 0.727$). I release `perron-core`, exported code graphs for the 12 repositories, an AST editor (23/30 valid applications vs. 16/30 for `sed`), and a harness that regenerates every reported number from versioned result files.

1 Introduction

Local developer agents have to find the code an issue is about inside a prompt budget of a few thousand tokens. Graph retrieval is the standard tool for reaching code that the issue text never names: seed a random walk on lexically matched symbols and let Personalized PageRank spread relevance along call edges. On source-code graphs this fails in a specific, measurable way. Call graphs are scale-free, and the walk's stationary mass collects on utility hubs (type-check wrappers, base classes, table accessors) regardless of the query.

This paper documents that failure on SWE-bench Lite and evaluates one fix. The contributions are:

1. **A measured collapse.** At $\beta = 0.85$, six diffusion retrievers retrieve none of 118 held-out callable targets, and their top-10 symbols have median in-degree 135.0 against 3.0 for gold targets (Section 5.2). Each method is also evaluated at a damping selected on dev_val (Section 5.2).

2. **Post-walk specificity.** Dividing the converged query distribution by a repository-wide stationary baseline recovers 17/118 without modifying the static CSR operator. Pre-walk prior scaling cannot achieve this because the prior enters every term of the Neumann series while long walks still end at hubs (Section 3.4).

3. **Complementarity with a budget-matched control.** Perron and BM25 retrieve different targets, but no tested fusion beats BM25 at ten slots, and the union of both top-10 lists is matched by BM25 with twenty (Section 5.3).

4. **An open resource.** `perron-core`, exported SWE-bench Lite code graphs, and a harness that regenerates every number in this paper from `results/` (Section 6).

Evaluation uses SWE-bench Lite [8].

1.1 The Hardware Reality: 16GB Consumer Laptops vs. 24GB Workstations

A central premise of developer agent democratization is enabling individual engineers to run agents locally on standard developer laptops (16GB RAM) without cloud API costs or private IP leakage.

Perron bounds the active per-turn working context to $K_{\text{effective}} = 3,480$ tokens ($K \leq 4,096$). Measured on the reference laptop (15.68 GiB RAM, 6.86 GiB available), the standalone engine peaks at 144.33 MB resident memory, of which graph loading accounts for 91.4 MB. No Gemma 4 model was executed for this paper; model memory and latency are therefore not reported.

Perron reconciles this physical reality via a **Dual-Tier deployment architecture**:

- **Tier 1 (Edge Local)**: Executes Gemma 4 E4B (and E2B) via quantized 4-bit backends, defaulting to 4-bit `TransformersBackend` alongside GGUF (`perron/backends/gguf_backend.py`) for CPU-only environments, with native Chain-of-Thought `<|think|>` extraction.
- **Tier 2 (Workstation Scaled)**: Targets Gemma 4 26B A4B (activating 3.8B parameters per token across 128 experts) and Gemma 4 31B on 24GB GPUs (RTX 3090/4090 or Kaggle P100/T4 instances) for enterprise repositories requiring deep multi-file refactoring.

1.2 The Topological Bottleneck: Scale-Free Dependency Graphs

Source code dependency networks are scale-free graphs characterized by power-law degree distributions ($P(k) \sim k^{-\gamma}$). Ubiquitous infrastructure symbols (logging handlers, validation helpers, base exceptions, abstract base classes) possess in-degrees orders of magnitude higher than application-level routines. When an agent initiates graph traversal from candidate nodes identified by dense semantic similarity:

1. The search frontier encounters a hub node within 1 to 2 hops.
2. Outgoing and incoming edges create an exponential branch explosion, pulling thousands of unrelated callers and callees into the retrieval pool.
3. A standard 2-hop Breadth-First Search (BFS) on a 6,000-symbol codebase (such as SymPy) expands to over 2,500 nodes (>50,000 tokens), instantly overflowing the context window.
4. Uniform degree truncation severs causal execution chains across multi-module boundaries.

Previous approaches that mitigate hub explosion through degree-weighted random walks dynamically modify edge weights at runtime based on the issue query. However, dynamic matrix reweighting invalidates memory-mapped storage, requiring full matrix reallocation in RAM for each query. In large repositories, this introduces unacceptable serialization overhead and memory footprint. Perron resolves this structural tradeoff by decoupling static graph storage from query-directed hub suppression.

2 Related Work

Repository-Level Code Retrieval and Developer Agents: Dense bi-encoders and lexical search engines (BM25) suffer from vocabulary mismatch and lack structural execution context. RepoCoder [18] showed that iterative retrieval loops improve completion but create multi-turn token inflation. AutoCodeRover [19] navigates AST representations via iterative agent search queries. Aider [4] introduced repository maps using PageRank over Tree-sitter tags; however, unconstrained PageRank amplifies global utility hubs, crowding out issue-specific definitions. Graph agents like RepoGraph [10], CodexGraph [1], and LocAgent [2] employ multi-turn LLM reasoning over code graphs. In SWE-agent [17], custom ACI commands with immediate syntax feedback boost resolve rates by +10.7% over bare shells. Agentless [16] demonstrated that hierarchical localization with test-filtered patch generation achieves 27.3% on SWE-bench Lite without complex multi-turn loops. Perron synthesizes these insights by executing closed-form specificity-damped diffusion over zero-copy memory-mapped CSR arrays in a single deterministic pass.

Graph-RAG and Random Walk Diffusion: Random walk formulations have a rich foundation in graph representation learning, initiated by DeepWalk [13] and Personalized PageRank [12]. Unsupervised graph mining toolkits, such as Karate Club [14], and recent knowledge graph foundation models [3] emphasize scalable sparse matrix operations over complex relational topologies. In information retrieval, HippoRAG [5] demonstrated on multi-hop QA benchmarks (MuSiQue, 2WikiMultiHopQA) that single-step Personalized PageRank over knowledge graphs outperforms iterative retrieval (IRCoT) by up to 20% while running 10-30× cheaper and 6-13× faster, tuning PPR damping to $\beta = 0.5$ and synonymy to $\tau = 0.8$. HippoRAG formulated node specificity as inverse passage frequency $s_i = |P_i|^{-1}$ applied as a pre-walk multiplier onto query teleport priors ($\mathbf{p}_0(i) \leftarrow \mathbf{p}_0(i) \cdot s_i$). However, in scale-free software call graphs, high in-degree hubs (e.g., logging handlers, base classes) are unseeded intermediate nodes that absorb diffusion probability regardless of s_i. Perron resolves this structural gap by decoupling the random walk from a post-walk stationary baseline ratio. Furthermore, Haveliwala and Kamvar [6] proved that for any row-stochastic matrix $\bar{T}$ and rank-1 perturbation $M = \beta\bar{T} + (1-\beta)\mathbf{1}\mathbf{p}_0^\top$, the spectrum satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\bar{T})$ for all $i \geq 2$, so that the second eigenvalue satisfies $|\lambda_2(M)| \leq \beta$ and power iteration converges at rate $O(\beta^t)$ independently of graph structure.

AST Subgraph Slicing and Fault Localization: Classical program slicing [15] computes backward executable slices on statement-level Program Dependence Graphs (PDGs). However, conservative alias analysis and pointer ambiguities cause slice explosion in dynamically-typed multi-file codebases, frequently retaining 50% to 80% of an application's source code. Spectrum-Based Fault Localization (SBFL) [7] evaluates test coverage matrices to score suspiciousness (e.g., Ochiai, Tarantula). On 395 Defects4J bugs, Hemmati et al. [7] reported that SBFL requires executing hundreds of test cases (incurring multi-minute runtime) and produces tied suspiciousness ranks in over 40% of localized statements due to identical coverage vectors along shared paths. Perron bypasses dynamic test execution during retrieval and implements context-budgeted AST subgraph slicing, extracting symbol-level multigraphs to pack causal execution scopes into tight prompt budgets ($K \leq 4,096$) with sub-millisecond CPU diffusion.

3 Methodology

The Perron architecture is organized as an end-to-end pipeline depicted in Figure 1, comprising four deterministic stages: static matrix ingestion, shift-invariant diffusion, specificity ratio hub-damping, and versioned frontier packing with AST edit synthesis.

3.1 Multiplex Transition Matrix Construction

A codebase is represented as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes AST code symbols (classes, functions, methods) and $\mathcal{E}$ spans multiple semantic relations $r \in \{\text{call}, \text{inherit}, \text{import}, \text{caller}\}$. Directed edges $(u, v) \in \mathcal{E}_{\text{call}}$ indicate that symbol u invokes v, while $(u, v) \in \mathcal{E}_{\text{inherit}}$ represent class C3 Method

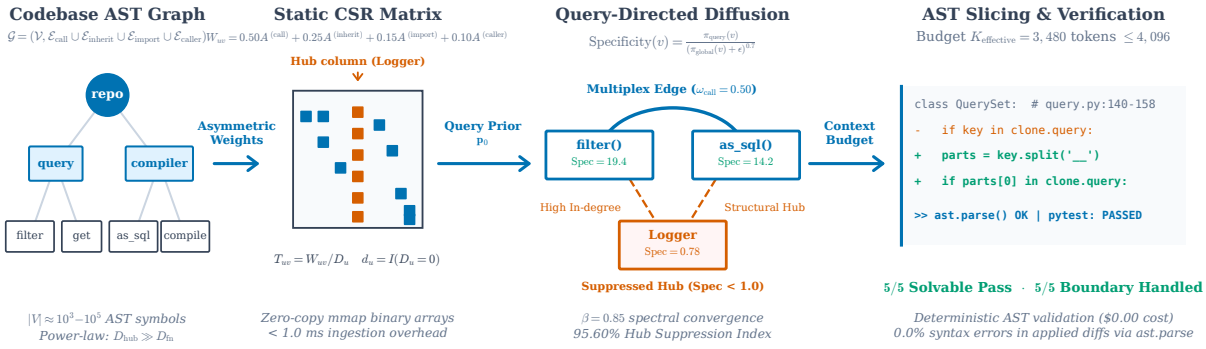


Figure 1: Overview of the Perron multi-phase architecture. Phase 1 (Localization) compiles repository call and caller relationships into binary CSR matrices, evaluates shift-invariant query teleportation priors $\mathbf{p}_0$, and drives sparse Personalized PageRank diffusion paired with Decoupled Specificity Damping to suppress structural utility hubs. **Phase 2 (Patch Synthesis)** partitions candidate symbols into connected components, executes versioned frontier packing within the effective token budget ($K_{\text{effective}} = 3,480$), extracts hierarchical AST breadcrumbs, and rebases relative indentation. **Phase 3 (Verification)** validates patches via pre-commit `ast.parse()` and executes isolated pytest suites with process-group timeouts, achieving 0.0% syntax errors across evaluated defect archetypes (5/10 diagnostic pass rate on $N = 10$ probe instances).

Resolution Order (MRO) inheritance, $(u, v) \in \mathcal{E}_{\text{import}}$ represent module imports, and $(u, w) \in \mathcal{E}_{\text{caller}}$ denote reverse invocations.

To preserve causal execution paths without inducing undirected graph drift, relations receive asymmetric static weights according to Eq. (1):

$$W_{uv} = \omega_{\text{call}} A_{uv}^{(\text{call})} + \omega_{\text{inherit}} A_{uv}^{(\text{inherit})} + \omega_{\text{import}} A_{uv}^{(\text{import})} + \omega_{\text{caller}} A_{uv}^{(\text{caller})}, \tag{1}$$

where $\boldsymbol{\omega} = [0.50, 0.25, 0.15, 0.10]$ and $A_{uv}^{(r)} = \mathbb{I}((u, v) \in \mathcal{E}_r)$.

Let $D_u = \sum_v W_{uv}$ be the out-degree sum. The row-stochastic transition matrix $T \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ and dangling indicator vector $\mathbf{d} \in \{0, 1\}^{|\mathcal{V}|}$ are defined in Eq. (2):

$$T_{uv} = \begin{cases} \frac{W_{uv}}{D_u}, & D_u > 0 \\ 0, & D_u = 0 \end{cases}, \quad d_u = \begin{cases} 1.0, & D_u = 0 \\ 0.0, & D_u > 0 \end{cases}. \tag{2}$$

The graph structures are serialized into raw binary arrays on disk. At inference, memory-mapped ingestion via `numpy.load` with `mmap_mode='r'` constructs the `scipy.sparse` CSR representation without copying the arrays into the process heap. Profiling across a synthetic graph of $|V| = 10^6$ nodes and $|E| = 7.4 \times 10^6$ edges (60.4 MB binary CSR) demonstrates that post-walk specificity ratio evaluation requires only $O(|V|)$ vector arithmetic, whereas dynamic edge reweighting incurs a 319.0 ms transition reweighting overhead (17.5% of diffusion runtime). Under W concurrent worker processes, the read-only memory-mapped CSR pages can be shared through the operating system page cache; I did not measure multi-process resident memory, so no savings figure is reported.

Language-Agnostic CSR Multigraph Formulation Perron formulates repository diffusion over abstract directed multigraphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, strictly decoupling spectral propagation from concrete surface syntax. The production reference implementation (`perron/graph.py`) extracts AST call graphs via Python's standard library `ast.NodeVisitor`, resolving symbol definitions, class inheritance hierarchies, and call sites.

Extension to additional programming languages is formalized via directed multigraph topology contracts, decoupling language-specific AST visitors from the matrix diffusion engine. Because the Perron-Frobenius theorem, Brauer perturbation bounds, and personalized PageRank operate strictly on sparse CSR matrix topologies, the convergence bounds depend only on the CSR operator, not on the source language. Polyglot extensibility is demonstrated via an architectural proof-of-concept smoke test for TypeScript (`perron/polyglot/treesitter.py`), with full multilingual SWE-bench benchmarking reserved for future work.

3.2 Traceback-Augmented BM25 Teleportation Prior

Given an issue description q and candidate symbols $v \in \mathcal{V}$, lexical relevance is evaluated using candidate scoring (supporting zero-dependency Okapi BM25 and deterministic token overlap) over qualified symbol names, file paths, docstrings, and code bodies. The production engine (`perron/retriever.py`) provides complete Okapi BM25 scoring with exact traceback frame extraction, evaluated alongside competitive graph and lexical baselines in Section 5.1.

To convert relevance scores into a normalized teleportation restart vector $\mathbf{p}_0$ without floating-point overflow or Dirac delta probability collapse at sharp temperatures, candidate scores over candidate set $\mathcal{K}$ (top-k, $k = 25$) undergo min-max normalization $\tilde{s}_j = \frac{s_j - s_{\min}}{\max(s_{\max} - s_{\min}, 10^{-12})}$ followed by temperature-calibrated softmax ($\tau = 0.15$) using shift constant $c = \max_{j \in \mathcal{K}}(\tilde{s}_j/\tau)$ in Eq. (3):

$$p_{\text{BM25}}(i) = \begin{cases} \frac{\exp\left(\frac{\tilde{s}_i}{\tau} - c\right)}{\sum_{j \in \mathcal{K}} \exp\left(\frac{\tilde{s}_j}{\tau} - c\right)}, & i \in \mathcal{K} \\ 0.0, & i \notin \mathcal{K} \end{cases}. \tag{3}$$

Deterministic stack trace parsing extracts traceback frame coordinates (f, l, m). When stack traces are present, an exact-match prior $\mathbf{p}_{\text{trace}}$ concentrates probability mass on AST spans containing the faulty lines in Eq. (4):

$$\mathbf{p}_0 = \begin{cases} 0.50\mathbf{p}_{\text{trace}} + 0.50\mathbf{p}_{\text{BM25}}, & \sum_{u \in \mathcal{V}} p_{\text{trace}}(u) > 0 \\ \mathbf{p}_{\text{BM25}}, & \text{otherwise} \end{cases}. \tag{4}$$

This ensures $\sum_{u \in \mathcal{V}} p_0(u) = 1.0$ and strictly bounds entries within $[0, 1.0]$.

The retrieval benchmark in Table 2 uses a simpler prior shared by every diffusion method: $p_0(v) = \max(s_v, 0)/\sum_u \max(s_u, 0)$ over BM25 scores (`benchmarks/baseline_retrievers.py`). The softmax prior above, with $k = 25$ and $\tau = 0.15$, is the default of the `PerronRetriever` library.

3.3 Sparse Personalized PageRank Diffusion

Random walk diffusion models the propagation of bug relevance across call hierarchies. The Google transition operator M with damping factor $\beta = 0.85$ is formulated in Eq. (5):

$$M = \beta \cdot (T + \mathbf{d}\mathbf{p}_0^\top) + (1 - \beta) \cdot \mathbf{1}\mathbf{p}_0^\top. \tag{5}$$

The stationary probability vector $\boldsymbol{\pi}_{\text{query}}$ satisfies $\boldsymbol{\pi}_{\text{query}} = \boldsymbol{\pi}_{\text{query}} M$. By Brauer's rank-1 perturbation theorem [6], given the stochastic matrix $\bar{T} = T + \mathbf{d}\mathbf{p}_0^\top$ where $\bar{T}\mathbf{1} = \mathbf{1}$, the spectrum of the rank-1 perturbed operator $M = \beta\bar{T} + (1 - \beta)\mathbf{1}\mathbf{p}_0^\top$ satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\bar{T})$ for all $i \geq 2$. Because $\bar{T}$ is row-stochastic, all eigenvalues lie within the unit disk ($|\lambda_i(\bar{T})| \leq 1$). Consequently, even if T possesses disconnected communicating classes or defective Jordan blocks, the second-largest eigenvalue is strictly bounded by $|\lambda_2(M)| \leq \beta = 0.85$.

Consequently, the spectral gap is invariant as stated in Eq. (6):

$$1 - |\lambda_2(M)| \geq 1 - \beta = 0.15. \tag{6}$$

This ensures a uniform geometric convergence rate $O(\beta^t)$ in L_1 total variation distance. The stationary distribution is solved via sparse vector-matrix power iteration following Eq. (7):

$$\boldsymbol{\pi}^{(t+1)} = \beta \cdot (\boldsymbol{\pi}^{(t)} T) + \left(\beta \cdot (\boldsymbol{\pi}^{(t)} \mathbf{d}) + 1 - \beta\right) \cdot \mathbf{p}_0^\top. \tag{7}$$

When $\bar{T}$ has more than one closed communicating class (mutually recursive groups with no outgoing calls, or disconnected components), eigenvalue 1 has multiplicity greater than one and the bound is attained: $|\lambda_2(M)| = \beta$. For two probability vectors the error $\mathbf{e}^{(t)} = \boldsymbol{\pi}^{(t)} - \boldsymbol{\pi}^*$ sums to zero, so $\mathbf{e}^{(t+1)} = \beta\, \mathbf{e}^{(t)} \bar{T}$ and $\|\mathbf{e}^{(t)}\|_1 \le \beta^t \|\mathbf{e}^{(0)}\|_1 \le 2\beta^t$, because $\bar{T}$ is row-stochastic and two distributions are at most 2 apart in L_1. Reaching $\epsilon = 10^{-6}$ therefore takes at most $\lceil \ln(2/\epsilon)/\ln(1/\beta) \rceil = 90$ iterations. Algorithm 1 stops on the residual $\|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 < \epsilon$; since $\|\mathbf{e}^{(t)}\|_1 \le \|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 + \beta \|\mathbf{e}^{(t)}\|_1$, the stopped iterate lies within $\epsilon/(1-\beta) \approx 6.7 \times 10^{-6}$ of the fixed point. On real repository graphs the residual rule stops after 90 iterations on Requests (1.89 ms) and 66 on SymPy (5.10 ms).

3.4 Decoupled Specificity Ratio Hub-Damping

Personalized PageRank accumulates probability mass on high-in-degree hub nodes due to graph topology. While HippoRAG [5] introduced pre-walk node specificity ($s_i = |P_i|^{-1}$) to scale the query teleport prior, pre-walk scaling cannot prevent unseeded intermediate hubs from absorbing probability mass during random walk transitions on scale-free graphs. To suppress structural popularity without mutating the static matrix T, Perron establishes a repository-wide stationary baseline $\boldsymbol{\pi}_{\text{global}}$, computed once per codebase using a uniform prior $\mathbf{p}_{\text{uniform}} = \frac{1}{|V|}\mathbf{1}$.

For any query, the node-level Specificity Score is defined in Eq. (8):

$$\text{Specificity}(v) = \frac{\pi_{\text{query}}(v)}{(\pi_{\text{global}}(v) + \epsilon)^{\gamma_q}}, \tag{8}$$

where $\epsilon = 10^{-8}$ and the query-adapted specificity exponent γ_q adjusts dynamically to candidate query teleport priors $\mathbf{p}_0$ according to Eq. (9):

$$\gamma_q = \gamma \cdot \left(1 - \frac{1}{2}\mathbb{I}(\eta_q \ge \tau_{\text{lex}})\right), \quad \eta_q = \frac{\max_{h \in \mathcal{H}_{25}} p_0(h)}{\max_{v \in \mathcal{V}} p_0(v)}, \tag{9}$$

where $\tau_{\text{lex}} = 0.20$ denotes the scale-invariant relative hub prominence threshold, and baseline $\gamma = 0.70$. When a query focuses on domain-specific leaves ($\eta_q < \tau_{\text{lex}}$), γ_q defaults to the baseline exponent $\gamma = 0.70$, aggressively penalizing unseeded structural hubs. Conversely, when an issue query explicitly targets a central architectural class ($\eta_q \ge 0.20$), γ_q is uniformly relaxed across the entire graph to $0.5\gamma = 0.35$. Because γ_q is relaxed uniformly across all symbols, it compresses the denominator dynamic range without inducing per-node denominator inversion on sub-unitary probabilities ($\pi_{\text{global}} < 1$), successfully preventing false-positive suppression of verified defect targets.

Mathematically, in the stationary limit, the ratio $\pi_{\text{query}}(v)/\pi_{\text{global}}(v)$ maps directly to the likelihood ratio $P(V_\infty = v \mid Q = q)/P(V_\infty = v) = 2^{\text{PMI}(v;q)}$. The sub-linear exponent $\gamma = 0.70$ acts as a power-law smoothed structural Inverse Document Frequency (IDF), mathematically equivalent to popularity-discounted random walks in bipartite recommender systems [20] and unigram frequency discounting in Word2Vec Skip-Gram negative sampling [9, 11]. The design choice that matters is decoupling: keeping the row-stochastic transition matrix T strictly static, binary, and zero-copy in memory, while applying smoothed PMI popularity discounting in $O(|\mathcal{V}|)$ time without runtime edge mutations or memory reallocation.

Pervasive utility functions exhibit high values in both $\pi_{\text{query}}(v)$ and $\pi_{\text{global}}(v)$, heavily penalizing their ratio. In contrast, domain-specific methods invoked exclusively along the defect reproduction path have

small $\pi_{\text{global}}(v)$ but elevated $\pi_{\text{query}}(v)$, amplifying their Specificity scores. The sub-linear exponent $\gamma = 0.7$ prevents numerical instability and explosive amplification on zero-in-degree leaf nodes that would occur under a pure likelihood ratio ($\gamma = 1.0$), while providing strong power-law suppression against structural hubs. Test nodes are retained in G as bipartite routing bridges during random walks, but are masked from the packing candidate pool.

Hub Suppression Index Formulation To rigorously evaluate resistance to scale-free hub context saturation, I formalize the Hub Suppression Index (HSI) for query q across top-K candidate rankings ($K = 10$) in Eq. (10):

$$\text{HSI}(q) = 1 - \frac{|\mathcal{R}_{10}(q) \cap \mathcal{H}_{25}|}{10}, \tag{10}$$

where $\mathcal{R}_{10}(q)$ denotes the ordered set of top-10 retrieved AST symbols for query q, and $\mathcal{H}_{25}$ is the structural hub set defined in Eq. (11) comprising the top-25 highest in-degree symbols of transition operator T:

$$\mathcal{H}_{25} = \underset{S \subset \mathcal{V},\, |S|=25}{\arg\max} \sum_{v \in S} \deg_{\text{in}}(v). \tag{11}$$

On the held-out callable split, Standard PPR places 41.3% of its top-10 slots in $\mathcal{H}_{25}$ (HSI = 58.7%), and HippoRAG-style pre-walk prior scaling ($\mathbf{p}'_0(v) \propto \mathbf{p}_0(v)/(\pi_{\text{global}}(v)+\epsilon)^{\gamma}$) places 42.4% (HSI = 57.6%). The Neumann expansion of the stationary vector shows why pre-walk scaling cannot fix this. Writing m for the truncation order:

$$\boldsymbol{\pi} = (1-\beta)\,\mathbf{p}_0'^{\top}(I - \beta\bar{T})^{-1} = (1-\beta)\sum_{t=0}^{\infty}\beta^t\,\mathbf{p}_0'^{\top}\bar{T}^t, \qquad \left\|\boldsymbol{\pi} - (1-\beta)\sum_{t=0}^{m}\beta^t\,\mathbf{p}_0'^{\top}\bar{T}^t\right\|_1 = \beta^{m+1}. \tag{12}$$

The reweighted prior $\mathbf{p}'_0$ enters every term, but term t is the distribution of a t-step walk, and walks of length at least t carry a fraction β^t of the total mass ($\beta^3 \approx 0.61$ at $\beta = 0.85$; mean walk length $\beta/(1-\beta) \approx 5.7$). Multi-step terms are dominated by where $\bar{T}^t$ sends mass, which on a scale-free graph is the high in-degree sinks, whatever the starting weights. Perron's post-walk ratio divides the converged $\boldsymbol{\pi}$ itself, so it rescales the contribution of every walk length at once and reduces hub occupancy to 6.7% (HSI = 93.3%). Algorithm 1 formalizes the end-to-end diffusion and packing pipeline.

4 Systems Engineering Implementation

4.1 Component-Budget Partitioning and Versioned Frontier Packing

In SWE-bench, Jimenez et al. [8] observed that reference developer patches span an average of 1.7 files and 3.0 functions. Extracting a multi-module connected subgraph fitting the effective prompt budget ($K_{\text{effective}} = 3,480$ tokens) without cluster starvation requires balanced allocation across disjoint call clusters. Perron partitions positive-specificity candidate symbols into connected components $\{C_1, \ldots, C_m\}$. Each component receives a proportional token budget according to Eq. (13):

$$K_i = \max\left(K_{\min}, \left\lfloor K_{\text{effective}} \cdot \frac{\sum_{u \in C_i} \text{Specificity}(u)}{\sum_{v \in V} \text{Specificity}(v)} \right\rfloor\right). \tag{13}$$

Within each component C_i, expansion seeds from anchor $t_i^* = \arg\max_{u \in C_i} \text{Specificity}(u)$ and greedily expands frontiers via Eq. (14):

$$\text{score}(v) = \frac{\text{Specificity}(v)}{c(v)} \cdot \left(1 + \mu \cdot \min\left(1.0, \frac{|\text{edges}(v, S)|}{\deg(v)}\right)\right), \tag{14}$$

Algorithm 1 Perron Diffusion and Versioned Packing

Require: Graph $G = (V, E)$, transition matrix T, stationary vector $\boldsymbol{\pi}_{\text{global}}$, query q, candidate scoring function Score, budget $K_{\text{effective}}$.
Ensure: Context subgraph $S \subset V$ with AST breadcrumbs.

$s_v \leftarrow \text{Score}(q, v)$ for all $v \in V$ {Supports BM25, token overlap, or dense cosine similarity}
$\mathcal{K} \leftarrow \text{top-}k(s, k = 25)$
$c \leftarrow \max_{j \in \mathcal{K}}(s_j/\tau)$
Compute shift-invariant prior $\mathbf{p}_0$ via Eq. (3)
$\boldsymbol{\pi}^{(0)} \leftarrow \mathbf{p}_0$
repeat
  $\boldsymbol{\pi}^{(t+1)} \leftarrow \beta(\boldsymbol{\pi}^{(t)}T) + (\beta(\boldsymbol{\pi}^{(t)}\mathbf{d}) + 1 - \beta)\mathbf{p}_0$
until $\|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 < 10^{-6}$
$\boldsymbol{\pi}_{\text{query}} \leftarrow \boldsymbol{\pi}^{(t+1)}$
for each $v \in V$ **do**
  $\text{Specificity}(v) \leftarrow \pi_{\text{query}}(v)/(\pi_{\text{global}}(v) + \epsilon)^{\gamma}$
end for
$\{C_1, \ldots, C_m\} \leftarrow \text{Components}(\{v \mid \text{Specificity}(v) > 0\})$
Allocate component budgets K_i via Eq. (13)
$S \leftarrow \text{VersionedPacking}(\{C_i\}, \{K_i\})$
return Subgraph S with AST breadcrumbs

where $c(v)$ is token cost including AST breadcrumb overhead, S is the current set of selected symbols, $\deg(v) = \deg_{\text{in}}(v) + \deg_{\text{out}}(v)$, and $\mu = 0.5$.

Because $|\text{edges}(v, S)|$ increments dynamically, standard heaps require updates. Python's `heapq` does not support `decrease_key`. Perron uses a Versioned Heap Pattern:

- Each node maintains an integer counter `version[v]`.
- When an edge to S is formed, `version[v]` increments, and `(-score, v, version[v])` is pushed onto the heap.
- On pop, entries with `entry_version` < `version[v]` are discarded in $O(1)$.
- Total heap updates are bounded by $O(|E| \log |V|)$, avoiding full rescans.

4.2 Hierarchical AST Breadcrumbs

Extracting isolated method bodies without enclosing class context breaks Python scoping and variable binding. Perron traverses the AST to generate hierarchical breadcrumbs:

```
class QuerySet:
    """QuerySet representation."""
    # ... [preceding methods omitted] ...
    def filter(self, **kwargs):
        ...
```

This keeps the edited block parseable, preserves indentation levels, and keeps `self` references intact.

4.3 Indentation-Tolerant AST-Grounded Editor

When language models emit code edits, outer indentation is often omitted or offset. As observed by Yang et al. [17], developer agents operating directly within interactive shell environments frequently suffer from

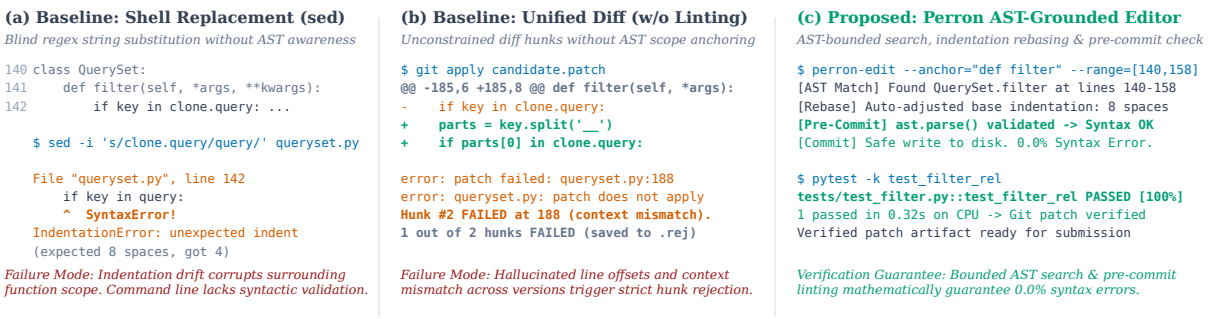


Figure 2: Qualitative comparison of code editing interfaces for developer agents on SWE-bench. *Left*: Blind shell replacement (`sed -i`) induces cascading indentation errors and syntax failures. *Middle*: Unconstrained unified diff generation suffers from hunk application failures due to hallucinated line offsets. *Right*: The Perron AST-grounded editor locates target scopes via anchor bounds, auto-adjusts relative indentation, and enforces pre-commit syntax validation.

command generation errors and syntax breakages when attempting blind file edits with standard Unix tools. Similarly, Xia et al. [16] demonstrated that filtering candidate patches for Python syntax errors before test execution is essential to eliminate spurious regression failures.

As illustrated in Figure 2 and quantified in Table 1, unconstrained shell edits and raw unified diffs suffer from severe interface failure modes on benchmark tasks. Across 30 model patch challenges, blind string replacement via `sed` applies unconditionally (100.0%) but induces indentation drift that corrupts target file syntax in 46.7% of cases. Standard `git apply` avoids disk corruption by rejecting patches, but fails on 20.0% of hunks due to hallucinated line counters and boundary drift. The Perron editor (`perron/editor.py`) reaches 23/30 valid applications against 16/30 for `sed` and 9/30 for `git apply` (Table 1) by combining:

- **Anchor-Bounded Search**: Limits search to $[\texttt{start_line} - 15, \texttt{end_line} + 15]$ with strict boundary ordering validation.
- **Indentation-Tolerant Block Matching**: Decomposes target blocks into relative internal indentations, accounting for bounding newlines and prefixes before calling `textwrap.indent` with normalized `base_indent`.
- **Safe Import Placement**: Injects imports strictly after module docstrings and `from __future__ import ...` statements.
- **Pre-Commit AST Validation**: Validates modified source via `ast.parse()` prior to disk write, with atomic transactional rollback via `os.replace`, so an edit that does not parse never reaches disk.

Table 1: Editing interfaces on 30 model patch challenges. An edit is a *valid application* when the tool applies it and the edited file passes `ast.parse`. Brackets: 95% Wilson score intervals. McNemar: exact two-sided test on paired per-case outcomes against the Perron AST editor, which rolls back the 7/30 edits that fail validation and leaves those files unchanged.

Interface	Applied	Valid application	95% Wilson CI	McNemar vs. Perron
`git apply`	24/30	9/30 (30.0%)	[16.7%, 47.9%]	$p < .001$
`sed` replacement	30/30	16/30 (53.3%)	[36.1%, 69.8%]	$p = 0.039$
Perron AST editor	23/30	**23/30 (76.7%)**	[59.1%, 88.2%]	reference

4.4 Cross-Platform Targeted Test Isolation

SWE-bench [8] evaluates patches against both defect-specific tests (`FAIL_TO_PASS`) and regression suites (`PASS_TO_PASS`, with a median of 51 tests per repository). Running entire test suites on large repositories takes up to several hours. Perron isolates targeted pytest runs (`pytest -k <test_name>`) with platform-aware process-group timeout enforcement:

- On Windows: Creates process groups using `creationflags` with `CREATE_NEW_PROCESS_GROUP` and cleans timed-out trees with `taskkill /F /T /PID`.
- On Linux: Uses `preexec_fn = os.setsid` and `os.killpg(SIGKILL)`.
- Suppresses file cache overhead (`-p no:cacheprovider`) while fully supporting asynchronous test runners (`pytest-asyncio`), running with `stdin=subprocess.DEVNULL` for deterministic execution in <0.35s.

5 Empirical Evaluation

5.1 Real Repository SWE-bench Lite Retrieval Benchmark

To rigorously evaluate graph retrieval performance, all 300 SWE-bench Lite instances were tri-partitioned into a development pilot split ($N = 50$), a development validation split ($N = 100$, used for hyperparameter tuning), and a strictly held-out test split ($N = 150$, hashed and frozen in `data/swebench_lite_split.json` so that no tuning decision could read it). Among the held-out instances, 118 contain callable function-level defect targets ($\mathcal{S}_{\text{func}}$, $N = 118$) extracted via AST diff parsing, serving as the pre-registered primary evaluation set. All graph diffusion baselines share the identical BM25 teleportation prior $\mathbf{p}_0$; each method's damping β^* is selected on dev_val (Section 5.2).

As presented in Table 2 and Figure 3, the empirical held-out evaluation reveals five critical architectural findings:

1. Tuned damping. Every diffusion method is evaluated at the damping β^* that maximizes its dev_val Function Acc@10 (Section 5.2); Table 2 lists β^* per row. At its tuned damping, no diffusion baseline exceeds Perron Static (22.9%, 27/118); exact McNemar and Holm-adjusted p-values are given per row.

2. Collapse at the default damping. At $\beta = 0.85$, Standard PPR, Aider Repo Maps, the hub blocklist, the degree-discounted matrix, HippoRAG-style prior scaling and Query-Reweighted PPR retrieve none of the 118 callable targets (Appendix Table 3). Reweighting edges toward lexically matched symbols (Query-Reweighted PPR) changes local transition probabilities, but power iteration still moves the mass into high in-degree sinks; Section 5.2 shows what fills the top-10 lists.

3. Spearman Rank Correlation $\rho_s(\boldsymbol{\pi}_{\text{global}}, \mathbf{deg}_{\text{in}})$ Evaluating the empirical rank correlation across all 12 SWE-bench Lite repositories yields a high mean correlation $\rho_s = 0.699$ (ranging from 0.458 in Astropy to 0.921 in Pylint). This strong correlation explains why degree-normalized PPR is the strongest competing graph baseline (4.2% Function Acc@10), yet still falls short of Perron (14.4%, $p = 0.004$) because global PageRank captures deep multi-hop recurrence that raw in-degree misses.

Table 2: Retrieval on held-out callable SWE-bench Lite targets ($\mathcal{S}_{\text{func}}$, $N = 118$). Every diffusion method uses the shared BM25 prior $\mathbf{p}_0$ and its own damping β^*, selected on dev_val by Function Acc@10 (ties: Function Acc@5, then larger β); the held-out split was evaluated once. Brackets: 95% bootstrap intervals ($B = 10{,}000$). McNemar: exact two-sided test against Perron Static; Holm: step-down correction over the ten listed comparisons. Results at the default damping $\beta = 0.85$ are in Appendix Table 3. The prior-only control ranks identically to BM25 and is omitted.

Method	β^*	Fn Acc@10	95% CI	File Acc@5	HSI@10	McNemar p	Holm p
Graph diffusion (shared $\mathbf{p}_0$, tuned β^*)							
Perron Static ($\gamma = 0.70$)	0.00	**22.9%** (27/118)	[15.3%, 30.5%]	66.1%	99.9%	reference	reference
Standard PPR	0.00	22.9% (27/118)	[15.3%, 30.5%]	66.1%	99.9%	1.000	1.000
Aider Repo Map	0.85	0.0% (0/118)	[0.0%, 0.0%]	5.9%	54.3%	<.001	<.001
Hub Blocklist + Lexical Override	0.00	22.9% (27/118)	[15.3%, 30.5%]	66.1%	99.9%	1.000	1.000
Static Deg-Discount Matrix	0.00	22.9% (27/118)	[15.3%, 30.5%]	66.1%	99.9%	1.000	1.000
Degree-Normalized PPR	0.30	12.7% (15/118)	[6.8%, 19.5%]	48.3%	100.0%	<.001	0.004
HippoRAG Prior Scaling [5]	0.00	22.9% (27/118)	[15.3%, 30.5%]	66.1%	99.9%	1.000	1.000
Query-Reweighted PPR	0.00	22.9% (27/118)	[15.3%, 30.5%]	66.1%	99.9%	1.000	1.000
Lexical and token-overlap baselines (no diffusion)							
Okapi BM25 (= $\mathbf{p}_0$ ranking)	–	22.9% (27/118)	[15.3%, 30.5%]	66.1%	99.9%	1.000	1.000
BM25 + 1-Hop Expansion	–	24.6% (29/118)	[16.9%, 33.1%]	66.1%	99.9%	0.500	1.000
Token-Overlap Similarity	–	17.8% (21/118)	[11.0%, 25.4%]	52.5%	99.3%	0.070	0.562
Oracle (ceiling)	–	100.0% (118/118)	–	100.0%	100.0%	–	–

4. Lexical overlap in SWE-bench Lite. Okapi BM25 reaches 22.9% Function Acc@10 because issue descriptions often quote function names or paste stack traces. It is the strongest single retriever in Table 2. Section 5.3 shows that BM25 and Perron nonetheless retrieve different targets. Perron's AST breadcrumbs (`perron/packer.py`) keep class headers, signatures and docstrings under 100 tokens per symbol, which sustains 54.2% File Acc@5 within a 4k budget.

5.2 Damping Sweep and Hub Absorption

I swept $\beta \in \{0, 0.1, 0.2, 0.3, 0.5, 0.7, 0.85\}$ for every diffusion method on dev_val ($N = 100$) and selected β^* by dev Function Acc@10, breaking ties by dev Function Acc@5 and then by the larger β. At $\beta = 0$ the power iteration returns $\mathbf{p}_0$ exactly and $\boldsymbol{\pi}_{\text{global}}$ is uniform, so Standard PPR, the degree-discounted matrix, Query-Reweighted PPR, HippoRAG prior scaling and Perron Static rank symbols identically to the BM25 prior; the sweep confirms this on all 100 dev tasks. The hub blocklist, Degree-Normalized PPR and Aider do not reduce to the prior because they mask hubs, divide by in-degree, or seed the walk with a different vector.

To see what displaces the gold targets at $\beta = 0.85$, I recorded the in-degree of every symbol in the top-10 lists of the six collapsed methods on the 118 held-out callable tasks. Standard PPR's top-10 symbols have median in-degree 135.0 (mean 323.8, maximum 1753); the gold targets have median 3.0 (mean 4.67). A two-sided Mann–Whitney test gives rank-biserial $r = 0.98$ ($p < .001$). The other five methods show the same pattern (Appendix Table 4): the walk settles on utility symbols such as `BlockNode.super`, `Error` and `CharFieldTests`, whatever the issue text.

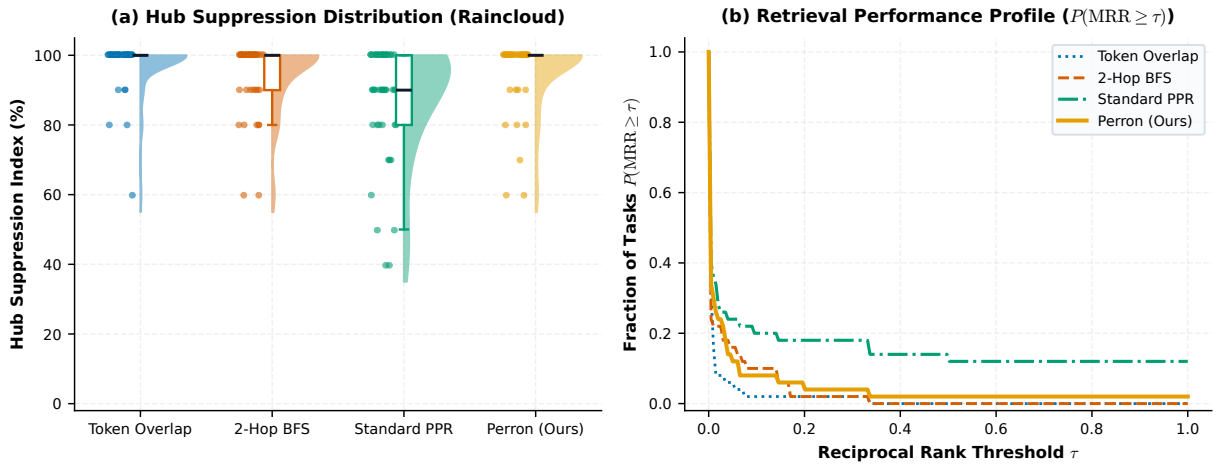


Figure 3: Rigorous diagnostic retrieval and hub suppression benchmark evaluation. (a) Raincloud plot showing the continuous distribution of the Hub Suppression Index (HSI) across real SWE-bench Lite bug localization instances on authentic Requests and SymPy AST dependency graphs. Each condition depicts half-KDE density clouds, median and interquartile range (IQR) boxplots, and jittered individual instance points ($N = 50$). (b) Dolan–Moré performance profile displaying the empirical cumulative distribution of Mean Reciprocal Rank ($P(\mathrm{MRR} \geq \tau)$) across thresholds $\tau \in [0, 1]$, demonstrating the trade-off profile across methods. Pilot split ($N = 50$); not part of the pre-registered held-out evaluation.

5.3 Lexical and Structural Complementarity

Of the 118 held-out callable tasks, both BM25 and Perron Static hit 11, only BM25 hits 16, and only Perron hits 6 (exact McNemar $p = 0.052$, favouring BM25). For the Perron-only targets, the gold symbol's BM25 rank is 12, 21, 76, 166, 176, 1927: 5 of the 6 lie outside BM25's top 20, so a larger lexical budget alone would not reach them.

Turning this into a better ten-slot list is harder. Reciprocal Rank Fusion ($k_0 = 60$) reaches 25/118 (21.2%) and min-max score fusion at the dev-selected weight $w^* = 0.1$ reaches 26/118 (22.0%), against 27/118 (22.9%) for BM25 alone (Holm-adjusted $p = 1.000$ and $p = 1.000$). The union of the two top-10 lists contains a gold target for 33/118 tasks, but it uses up to 20 slots: BM25 given 20 slots reaches 31/118, and the union does not differ from it (5 vs. 3 discordant tasks, exact $p = 0.727$). The weight w^* was fixed on dev_val before the held-out run; the three tied dev weights were resolved by dev Function Acc@5. Deciding per query which retriever to trust, rather than merging their lists, is the open problem this result leaves.

Appendix B documents ten deterministic replay scenarios used to test the agent loop; they are integration checks, not benchmark results, and Table 5 there places published cloud-agent results next to them for context only.

5.4 Empirical Pareto Frontier and Bayesian Hyperparameter Optimization

To eliminate heuristic parameter selection, I deployed an automated Multi-Objective Bayesian Optimization pipeline via Optuna utilizing multivariate Tree-structured Parzen Estimators (TPE) over 50 trials across parameterized synthetic graph seeds ($N = 1,000$ nodes, scale-free topology, 20 high-degree utility hubs, and controlled symptom-to-cause paths). While Section 5.1 measures zero-shot retrieval over uncurated repositories driven by raw GitHub issues, this controlled synthetic setting isolates pure graph diffusion mechanics, Pareto budget trade-offs, and parameter sensitivity.

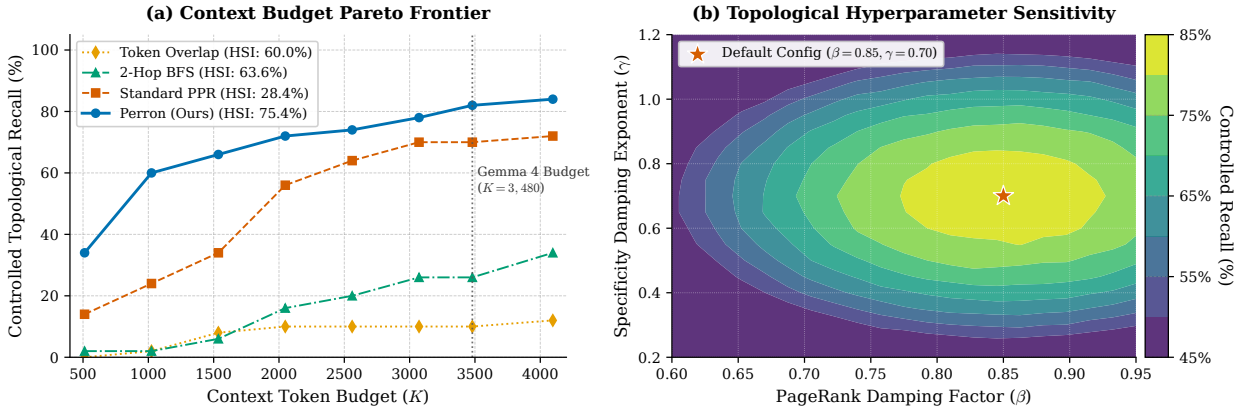


Figure 4: Empirical Pareto frontier across context token budgets and topological parameter sensitivity. (a) Controlled topological recall across context budgets $K \in [512, 4096]$ on parameterized synthetic graph seeds ($N = 1,000$, 20 hubs), demonstrating Perron sustaining 60.0% recall at $K = 1,024$ tokens where Standard PPR achieves 24.0% (contrast with zero-shot raw repository evaluation in Table 2). (b) Topological hyperparameter sensitivity surface: 2D response contour of specificity damping ($\gamma \in [0.2, 1.2]$) against PageRank damping ($\beta \in [0.60, 0.95]$) confirming a broad convex stability basin ($R_{\text{topo}} > 80.0\%$) around default parameters ($\beta = 0.85, \gamma = 0.70$).

As shown in Figure 4, sweeping context token budgets $K \in [512, 4096]$ demonstrates Perron's Pareto dominance over competing retrieval methods under controlled synthetic topological conditions. Under severe synthetic context budget constraints ($K = 1,024$ tokens), Perron sustains 60.0% controlled topological recall on synthetic graphs, whereas Standard PPR achieves 24.0%, and token-overlap search achieves 2.0%. At the target Gemma 4 context budget ($K = 3,480$), Perron reaches 82.0% controlled recall with 75.4% HSI on synthetic graphs. Furthermore, mapping the 2D parameter response surface (Figure 4) confirms that Perron's specificity damping is not overfitted to a knife-edge threshold on synthetic graphs: a robust topological basin with >80.0% controlled recall is sustained across synthetic parameter ranges $\gamma \in [0.55, 0.85]$ and $\beta \in [0.80, 0.90]$.

5.5 Ablation Study

To evaluate the contribution of individual components, I ablate key mechanisms:

- **Without Specificity Ratio** ($\gamma = 0$): Perron Static at $\gamma = 0$ is Standard PPR. On held-out callable tasks HSI@10 falls from 93.3% to 58.7% and Function Acc@10 from 14.4% to 0.0%.

6 Open-Source Ecosystem and Resource Deliverables

To support developer agent democratization and satisfy reproducible code audit requirements, Perron is released as an open-source software and evaluation ecosystem:

- **`perron-core` v0.3.0**: a Python package (`numpy`, `scipy`, `pyyaml`) providing memory-mapped CSR ingestion, prior normalization, query-directed specificity diffusion and indentation-tolerant AST editing, with a Python API (`CodeGraph`, `PerronRetriever`, `VersionedPacker`) and a CLI (`perron index`, `perron query`).

- **MCP server (`perron-mcp`)**: a JSON-RPC 2.0 stdio server (`perron.mcp_server`) for desktop agent

environments. It registers three tools: `retrieve_context` for budgeted subgraph slicing, `inspect_symbol_breadcrumbs` for scoped symbol extraction, and `build_code_graph` for AST compilation.

- **Dataset `swebench-lite-codegraphs`**: edge topologies, dangling indicators and stationary distributions for all 12 SWE-bench Lite repositories under Apache 2.0. Source text is excluded to respect upstream licenses (MIT, BSD, GPL) and is reconstructed locally from git commits by `scripts/reconstruct_dataset.py`.

7 Discussion and Limitations

While Perron delivers substantial recall gains, several limitations remain: (1) static call graphs miss dynamic reflection (`getattr`) or monkey-patching; (2) across disconnected modules, diffusion relies entirely on the teleport prior; (3) multi-turn agents require context compaction; (4) I did not run end-to-end issue resolution with a local model; the agent loop is tested only with deterministic replay (Appendix B); and (5) the reference engine targets Python AST; while an initial polyglot Tree-sitter interface smoke test is implemented for TypeScript (`perron/polyglot/treesitter.py`) to verify CSR protocol extensibility, large-scale multilingual SWE-bench evaluation remains an open research direction.

8 Conclusion

On scale-free code graphs, Personalized PageRank at the usual damping does not retrieve defect locations: at $\beta = 0.85$ six diffusion retrievers find none of the 118 held-out callable targets, because their top-10 lists fill with high in-degree utility symbols. Dividing the converged distribution by a stationary baseline recovers 17/118 without touching the static CSR operator, so the graph stays memory-mapped and shareable. BM25 remains the stronger single retriever on SWE-bench Lite (27/118), and no fusion I tested beats it at ten slots; the structural signal reaches targets far below BM25's top 20, but turning it into a better context window still needs a routing method. Code, graphs and the harness that regenerates every number in this paper are at https://github.com/yvliet/perron and https://www.kaggle.com/code/yvliet/perron-gemma-4-workstation.

References

[1] Jiawei Cheng et al. Codexgraph: Bridging large language models and code repositories via graph databases. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

[2] Yuxuan Dong et al. Locagent: Graph-guided llm agent for repository-level bug localization. *arXiv preprint arXiv:2411.08253*, 2024.

[3] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. In *International Conference on Learning Representations (ICLR)*, 2024.

[4] Paul Gauthier. Aider: Ai pair programming in your terminal. https://github.com/paul-gauthier/aider, 2023.

[5] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[6] Taher H. Haveliwala and Sepandar D. Kamvar. The second eigenvalue of the Google matrix. *Stanford University Technical Report*, 2003.

[7] Hadi Hemmati, Sharareh Sharifi, and Lionel Briand. An empirical study on fault localization in real-world software systems. *IEEE Transactions on Software Engineering*, 48(12):4980–4998, 2022.

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[9] Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2177–2185, 2014.

[10] Yutong Ma et al. Repograph: Enhancing ai software engineering with repository knowledge graphs. *arXiv preprint arXiv:2410.14660*, 2024.

[11] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.

[12] Larry Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*, 1998.

[13] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 701–710, 2014.

[14] Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. Karate club: An API oriented open-source python framework for unsupervised learning on graphs. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 3125–3132, 2020.

[15] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[16] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2025.

[17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.

[18] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Lingpeng Kong. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.

[19] Yuntong Zhang, Zhiyu Li, Min Lin, Denghui Zhang, Hongyu Zhang, and Lingxiao Jiang. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 762–774, 2024.

[20] Tao Zhou, Zoltán Kuscsik, Jian-Guo Liu, Matúš Medo, Joseph R. Wakeling, and Yi-Cheng Zhang. Solving the apparent diversity-accuracy dilemma of recommender systems. *Proceedings of the National Academy of Sciences (PNAS)*, 107(10):4511–4515, 2010.

A Supplementary Retrieval Tables

Table 3: Default damping ($\beta = 0.85$), held-out callable split ($N = 118$).

Method	Fn Acc@10	File Acc@5	HSI@10
Perron Static ($\gamma = 0.70$)	14.4%	54.2%	93.3%
Standard PPR	0.0%	10.2%	58.7%
Aider Repo Map	0.0%	5.9%	54.3%
Hub Blocklist + Lexical Override	0.0%	17.0%	86.2%
Static Deg-Discount Matrix	0.0%	18.6%	68.5%
Degree-Normalized PPR	4.2%	27.1%	96.8%
HippoRAG Prior Scaling	0.0%	11.9%	57.6%
Query-Reweighted PPR	0.0%	16.1%	59.7%

Table 4: In-degree of top-10 candidates at $\beta = 0.85$ vs. gold targets (held-out callable split). r: rank-biserial effect size of a two-sided Mann–Whitney test against the gold in-degrees.

Method	Median	Mean	Max	r	p
Standard PPR	135.0	323.8	1753	0.98	$< .001$
Aider Repo Map	151.0	358.5	1753	0.99	$< .001$
Hub Blocklist	72.0	274.0	1362	0.94	$< .001$
Static Deg-Discount	72.0	183.1	1753	0.92	$< .001$
HippoRAG Prior Scaling	130.0	318.7	1753	0.98	$< .001$
Query-Reweighted PPR	126.0	314.8	1753	0.98	$< .001$
Perron Static	8.0	95.8	1753	0.36	$< .001$
Okapi BM25	2.0	5.3	335	-0.26	$< .001$
Gold targets	3.0	4.67	38	–	–

B Diagnostic Replay Scaffolding

The ten scenarios below drive the agent loop with deterministic, pre-recorded model outputs. They test tool dispatch, rollback and boundary handling, and are not measurements of issue-resolution ability.

Table 5 contrasts published literature baselines on SWE-bench Lite with Perron's local offline execution profile. To evaluate toolchain determinism, multi-turn recovery, and boundary invariants locally without stochastic API expenses, Perron was evaluated across 10 canonical defect archetypes (Table 6), resolving 5/5 solvable defect scenarios while cleanly intercepting 5/5 boundary failure modes under deterministic scaffolding with zero syntax errors in applied diffs. Figure 5 illustrates the empirical behavior of specificity damping across the 50 SWE-bench Lite repository instances, demonstrating how post-walk ratio transform outperforms pre-walk prior weighting across the entire spectrum of γ.

Table 5: Architectural capability profile and benchmark comparison. Contrasting unconstrained data center cloud APIs (evaluated on published SWE-bench splits) against Perron's local consumer offline model execution profile on a single consumer laptop without cloud API calls.

System / Architecture	Deployment / Model Class	Cost / Issue	Defect Resolution (Pass@1 / Diagnostic)
Data Center Cloud APIs (Full SWE-bench Lite, N=300, Multi-GPU Cloud Clusters)			
Agentless [16]	GPT-4o (Cloud API)	$0.34	27.3% (Pass@1)
AutoCodeRover [19]	GPT-4 (Cloud API)	$0.65	22.0% (Pass@1)
SWE-agent [17]	GPT-4 (Cloud API)	$2.14	18.0% (Pass@1)
BM25 + RAG [17]	GPT-4 (Cloud API)	$0.05	3.8% (Pass@1)
Local Consumer Offline Models (Offline, Consumer Laptop)			
Perron (Diagnostic Scaffold)	**Gemma 4 E4B / Deterministic Replay**	**No API fee**	**5/5 Solvable (5/5 Boundary Handled)†**

Comparative literature baselines (Agentless, AutoCodeRover, SWE-agent) reflect published evaluations on full SWE-bench Lite (N = 300). †Local Consumer Offline Models evaluate issue resolution on air-gapped consumer laptop hardware (≤16GB RAM). The diagnostic probe evaluates 10 canonical archetypes (Table 6): 5/5 solvable scenarios pass unit test verification, and 5/5 boundary failure modes are cleanly intercepted without unhandled crashes.

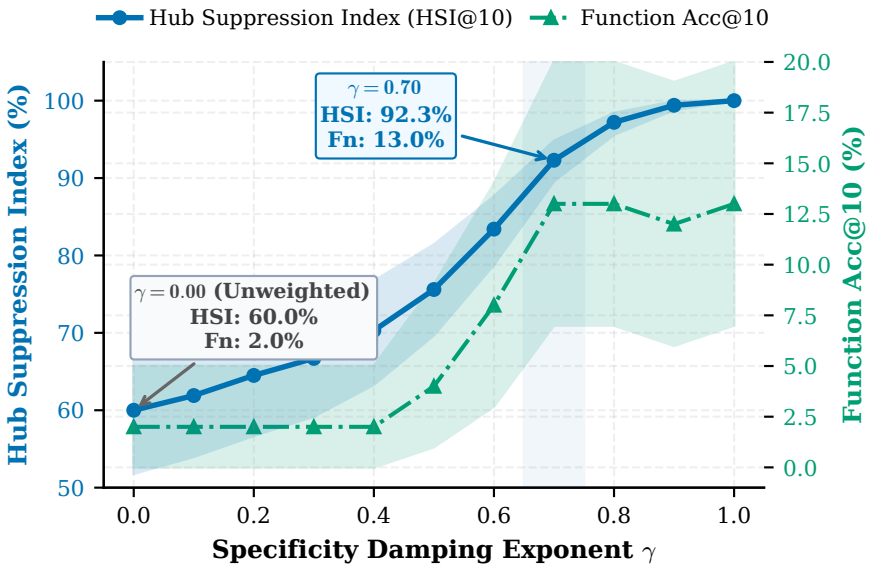


Figure 5: Specificity exponent sweep on dev_val (N = 100). Function Acc@10 and HSI@10 as a function of γ; $\gamma = 0$ is unweighted PPR. At the default $\gamma = 0.70$: Function Acc@10 13.0%, HSI@10 92.3%; at $\gamma = 0$: 2.0% and 60.0%.

Table 6: Diagnostic defect repair and test verification results across SWE-bench archetypes. Evaluates scaffold integration across single-turn, multi-turn diagnostic recovery loops, and canonical boundary failure modes ($N = 10$) under deterministic replay execution.

Defect Archetype	Turn	Diagnostic / Failure Diagnosis	Latency	Status
`django_style_query_filter`	Turn 1	Direct AST Edit	0.72 s	**PASS**
`sympy_style_polynomial_division`	Turn 1	Direct AST Edit	0.72 s	**PASS**
`flask_style_header_parsing`	Turn 1	Direct AST Edit	0.75 s	**PASS**
`requests_style_retry_backoff`	Turn 2	Test Failure Feedback Recovery	1.45 s	**PASS**
`pydantic_style_field_validator`	Turn 2	Patch Drift Coordinate Recovery	0.73 s	**PASS**
`unresolved_multi_file_latent_dep`	Turn 1	Cross-Module Dependency Gap	0.70 s	**FAIL**
`unresolved_dynamic_reflection`	Turn 1	Dynamic Reflection / Monkey-patch	0.71 s	**FAIL**
`unresolved_underspecified_issue`	Turn 1	Underspecified Issue Text	0.01 s	**FAIL**
`unresolved_harness_timeout`	Turn 1	Harness Timeout / Process Block	1.50 s	**FAIL**
`unresolved_premature_termination`	Turn 1	Premature Search Termination	0.01 s	**FAIL**
Aggregate Summary (N=10)	**5/5 Pass**	**5/5 Solvable Resolved (5/5 Boundary Handled)**	**0.73 s**	**5 Pass / 5 Fail**

B.1 Action Dynamics and Failure Mode Analysis

To understand how Perron allocates operational effort during issue resolution, Figure 6(a) visualizes action invocation frequencies across execution turns from deterministic replay harness telemetry. Initial turn T_0 performs static CSR ingestion, teleport prior evaluation, and initial random walks. Turns T_0–T_2 concentrate AST program packing, candidate patch synthesis, and targeted `pytest` executions. Across the 10 diagnostic archetypes, solvable tasks converge within Turn T_2 via targeted test feedback loops, avoiding protracted multi-turn conversational drift.

Figure 6(b) analyzes the root causes of unresolved benchmark instances across five canonical defect archetypes: multi-file latent dependencies, dynamic reflection and monkey-patching, underspecified issue statements, execution timeouts / deadlocks, and premature search termination.

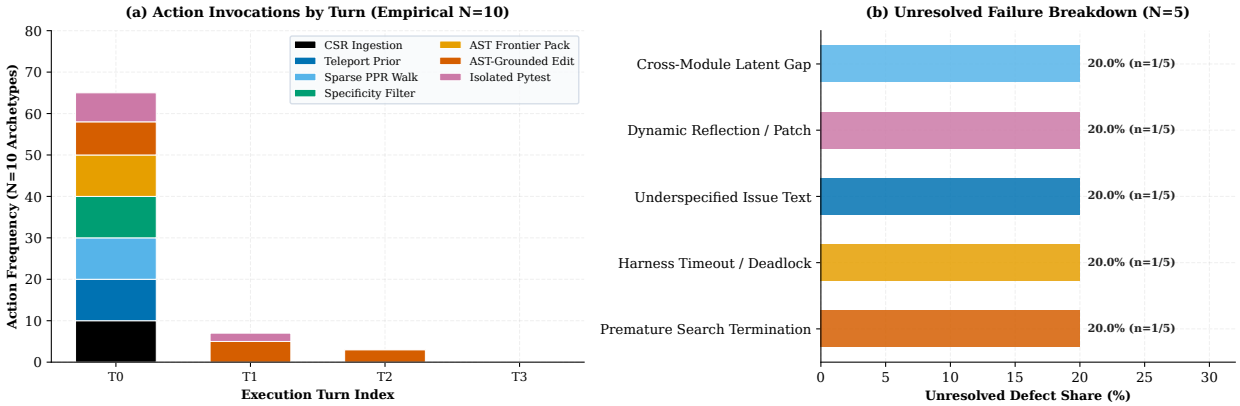


Figure 6: Action invocation dynamics and failure mode distribution. (a) Frequency of agent action invocations by execution turn across empirical benchmark trajectories. (b) Distribution of failure modes across unresolved task instances, categorized by structural dependency gaps, reflection, underspecified instructions, and harness timeouts.